

Métodos de Ingeniería de Software

Evaluación 2 (2026 - 1)

1. Descripción del trabajo

Los alumnos, en forma **personal**, deben desarrollar y desplegar una aplicación web diseñada en base a una arquitectura orientada a microservicios.

2. Lineamientos generales

- La evaluación se realizará en forma "**personal**".
- Para la evaluación no se debe entregar ningún informe escrito.
- Cada alumno debe presentarse en forma puntual en la fecha/hora programada. En caso contrario se le calificará con la nota mínima 1.0
- A la evaluación solamente deben presentarse aquellos alumnos que fueron planificados para la fecha. No se permitirá el ingreso de otros alumnos.

3. Acerca del proyecto de software

3.1 Contexto del problema

TravelAgency, una agencia dedicada a la comercialización de paquetes turísticos nacionales e internacionales, busca modernizar su modelo de negocio mediante el desarrollo de una plataforma web que permita a los clientes consultar y comprar viajes de forma autónoma. Actualmente, gran parte de sus operaciones se realizan de manera manual o asistida, utilizando planillas, correos electrónicos y sistemas no integrados, lo que genera limitaciones en la gestión y comercialización de sus servicios.

En el contexto actual, los clientes demandan acceso inmediato a información clara, actualizada y disponible en línea sobre destinos, precios, fechas y servicios incluidos. Sin embargo, la agencia no cuenta con una plataforma centralizada que permita explorar la oferta, comparar opciones y realizar reservas directamente desde la web. Esto provoca retrasos en la atención, dependencia de los agentes y pérdida de oportunidades de venta, especialmente fuera del horario laboral.

Además, la gestión manual genera inconsistencias en la disponibilidad de cupos, errores en la confirmación de reservas y dificultades en el control de pagos. La falta de integración entre los procesos de clientes, paquetes, reservas y pagos impide contar con una visión unificada del negocio, afectando tanto la eficiencia operativa como la experiencia del usuario.

Para abordar estas problemáticas, TravelAgency propone desarrollar una aplicación web que centralice la información y automatice los procesos clave. A través de esta plataforma, los usuarios podrán registrarse, buscar paquetes, visualizar información detallada, realizar reservas y efectuar pagos.

El sistema permitirá mejorar la eficiencia operativa, reducir errores y generar información estratégica mediante reportes, mientras que para los clientes ofrecerá una experiencia digital ágil, transparente y accesible, acorde a las exigencias actuales del mercado.

3.2 Principales actividades del modelo de negocio

3.2.1. Gestión de usuarios y clientes (Épica 1)

La plataforma debe permitir que los usuarios se registren y creen una cuenta personal para acceder a los servicios ofrecidos por la agencia. Durante este proceso, el usuario debe proporcionar información básica que permita su identificación y contacto, como nombre completo, correo electrónico, teléfono y, en algunos casos, información adicional relevante para viajes, como documento de identidad y nacionalidad.

Una vez registrado, el usuario podrá autenticarse en el sistema mediante credenciales seguras y acceder a funcionalidades personalizadas. La plataforma debe permitir la actualización de datos personales y la gestión de su perfil.

Esta actividad también contempla la gestión de distintos tipos de usuarios dentro del sistema, como clientes y administradores, permitiendo asignar permisos diferenciados según el rol.

Reglas de negocio

- El correo electrónico del usuario debe ser único dentro del sistema.
- No se puede registrar un usuario sin nombre completo, correo electrónico y contraseña.
- La contraseña debe cumplir una longitud mínima y criterios básicos de seguridad.
- El sistema debe validar el formato correcto del correo electrónico antes de registrar al usuario.
- Un usuario solo puede iniciar sesión si su cuenta se encuentra activa.
- Después de varios intentos fallidos consecutivos de autenticación, la cuenta puede quedar temporalmente bloqueada.
- Un cliente solo puede modificar sus propios datos personales.
- No se debe eliminar físicamente la cuenta de un cliente con historial de reservas; en su lugar, debe marcarse como inactiva.

3.2.2. Publicación y gestión de paquetes turísticos (Épica 2)

La plataforma debe permitir a la agencia definir y administrar su catálogo de paquetes turísticos, los cuales representan los productos que serán ofrecidos a los clientes a través del sitio web. Esta actividad implica la creación, edición y mantenimiento de la información asociada a cada paquete.

Cada paquete debe contener información completa y estructurada, incluyendo nombre, destino, descripción detallada, fechas disponibles, duración, precio, servicios incluidos, condiciones, restricciones y cupos disponibles. Además, se deben considerar atributos que permitan clasificar los paquetes, como tipo de viaje, temporada o categoría.

El sistema debe permitir modificar esta información de manera controlada, asegurando que los cambios sean consistentes y no afecten reservas ya realizadas. También debe permitir gestionar el estado del paquete, habilitando o deshabilitando su visibilidad en la plataforma según su disponibilidad o vigencia.

Reglas de negocio

- Todo paquete turístico debe tener nombre, destino, descripción, fecha de inicio, fecha de término, precio y cupos definidos.
- El precio del paquete debe ser mayor que cero.
- La fecha de término debe ser posterior a la fecha de inicio.

- Los cupos totales del paquete deben ser mayores que cero.
- Un paquete no puede publicarse como disponible si no tiene cupos.
- Un paquete con reservas asociadas no debe eliminarse físicamente; solo puede cambiar su estado.
- El sistema debe permitir estados controlados para el paquete, por ejemplo: disponible, agotado, no vigente o cancelado.
- Si un paquete ya tiene reservas registradas, no deben modificarse campos críticos que afecten la consistencia de la operación sin validación previa, como fechas base o cupos totales menores al número ya reservado.

3.2.3. Exploración y búsqueda de paquetes por parte del cliente (Épica 3)

La plataforma debe ofrecer una interfaz que permita a los usuarios navegar y explorar los paquetes turísticos disponibles de manera intuitiva. Esta actividad se centra en proporcionar mecanismos de búsqueda y filtrado que faciliten la identificación de opciones acordes a las preferencias del cliente.

Los usuarios deben poder realizar búsquedas utilizando distintos criterios, como destino, fechas de viaje, rango de precios, duración o tipo de experiencia. El sistema debe procesar estas consultas y presentar resultados relevantes, mostrando información resumida de cada paquete junto con la posibilidad de acceder a su detalle completo para visualizar información clara sobre lo que incluye cada paquete, evitando ambigüedades que puedan generar errores en la decisión de compra.

Reglas de negocio

- Solo deben mostrarse en las búsquedas los paquetes que estén en estado publicable para clientes.
- Un paquete agotado o no vigente no debe aparecer como opción reservable.
- El sistema debe filtrar resultados considerando únicamente paquetes con fechas válidas respecto del momento de consulta.
- Si el cliente aplica filtros, el sistema debe devolver solo paquetes que cumplan simultáneamente con los criterios ingresados.
- El precio mostrado al cliente debe corresponder al precio vigente almacenado en el sistema.
- La disponibilidad mostrada debe corresponder a los cupos realmente disponibles al momento de la consulta.
- El detalle del paquete debe mostrar claramente servicios incluidos y restricciones relevantes antes de iniciar la reserva.

3.2.4. Proceso de reserva en línea (Épica 4)

Una vez que el cliente ha seleccionado un paquete turístico, la plataforma debe permitir iniciar el proceso de reserva, registrando formalmente la intención de compra. Esta actividad implica asociar al usuario con el paquete elegido y capturar la información necesaria para la operación.

Durante este proceso, el sistema debe solicitar la cantidad de pasajeros, validar la disponibilidad de cupos en tiempo real y registrar los datos asociados a la reserva. También puede requerirse información adicional, como datos de los acompañantes, solicitudes especiales o preferencias del cliente.

El sistema debe gestionar estados de la reserva, comenzando típicamente en un estado inicial, como pendiente, el cual puede cambiar posteriormente según el avance del proceso de pago.

Asimismo, es importante asegurar la consistencia de los datos para evitar situaciones como la sobreventa de cupos o la generación de reservas inválidas.

Reglas de negocio

Reglas operativas

- Solo un usuario autenticado puede registrar una reserva.
- Una reserva debe estar asociada obligatoriamente a un cliente y a un paquete turístico existente.
- La cantidad de pasajeros debe ser mayor que cero.
- No se puede generar una reserva si la cantidad solicitada excede los cupos disponibles del paquete.
- Al crearse una reserva válida, el sistema debe descontar los cupos correspondientes del paquete.
- Toda reserva debe tener un identificador único.
- El monto total de la reserva debe calcularse en función del precio vigente del paquete y la cantidad de pasajeros.
- La reserva debe crearse con un estado inicial controlado, por ejemplo: pendiente de pago.
- No se puede registrar una reserva para un paquete cancelado, no vigente o agotado.
- Si la reserva expira por falta de pago en el plazo definido por el negocio, los cupos deben liberarse nuevamente.

Reglas comerciales

- **Descuento por cantidad de personas**
 - Si la cantidad de pasajeros supera un umbral definido (por ejemplo, ≥ 4 personas), se debe aplicar un descuento porcentual sobre el total de la reserva.
 - El porcentaje de descuento debe ser configurable por el sistema (por ejemplo, 5%, 10%).
 - El descuento debe aplicarse antes del cálculo final del monto a pagar.
- **Descuento por cliente frecuente**
 - Un cliente es considerado frecuente si cumple una condición definida (por ejemplo, ≥ 3 reservas históricas pagadas).
 - A los clientes frecuentes se les debe aplicar un descuento automático sobre nuevas reservas.
 - El descuento por cliente frecuente no puede ser modificado por el usuario y debe ser gestionado por el sistema.
- **Descuento por compra de múltiples paquetes**
 - Si un cliente realiza la compra de más de un paquete turístico en una misma operación o dentro de un período definido, se debe aplicar un descuento adicional.
 - El sistema debe identificar si las reservas están asociadas a la misma sesión de compra o a un mismo cliente dentro del período definido.
- **Acumulación y límites de descuentos**
 - El sistema debe definir si los descuentos son acumulables o excluyentes.
 - En caso de ser acumulables, debe existir un límite máximo de descuento permitido (por ejemplo, 20% del total).
 - El monto final de la reserva nunca puede ser negativo.
- **Promociones por tiempo limitado**
 - El sistema debe permitir definir promociones activas dentro de un rango de fechas.
 - Solo se deben aplicar descuentos si la reserva se realiza dentro del período de vigencia de la promoción.

- **Cálculo del monto final**
 - El sistema debe calcular el monto total considerando:
 - precio base del paquete
 - cantidad de pasajeros
 - descuentos aplicables
 - El monto final debe mostrarse claramente al usuario antes de confirmar la reserva.
- **Transparencia de descuentos**
 - El sistema debe detallar al usuario los descuentos aplicados y su origen (por ejemplo: "descuento por grupo", "cliente frecuente").
 - El usuario debe visualizar el precio original y el precio final antes de confirmar la reserva.

3.2.5. Gestión de pagos en línea (Épica 5)

La plataforma debe permitir a los clientes realizar el pago de sus reservas mediante un mecanismo de pago simulado, que represente el comportamiento básico de un sistema de pagos electrónicos sin requerir integración con servicios externos.

Esta actividad tiene como objetivo modelar el flujo de pago dentro del sistema, incluyendo la selección del medio de pago, el ingreso de datos básicos de una tarjeta de crédito simulada, el registro de la transacción y la actualización del estado de la reserva.

Durante el proceso de pago, el sistema debe permitir al usuario ingresar información representativa de una tarjeta de crédito, como número de tarjeta, fecha de expiración y código de seguridad (CVV), únicamente con fines de simulación. No se realizará validación contra entidades externas ni procesamiento real de pagos.

El sistema debe asumir que todo pago realizado es exitoso, permitiendo simplificar el flujo y enfocarse en la implementación de la lógica de negocio asociada a las reservas. Una vez confirmado el pago, se debe registrar la transacción y actualizar el estado de la reserva a pagada o confirmada.

Asimismo, el sistema debe operar únicamente con pagos totales, es decir, el cliente debe pagar el monto completo de la reserva en una sola transacción. No se contempla el manejo de pagos parciales ni cuotas.

Todo el flujo debe simular de manera coherente el comportamiento de una pasarela de pago real, permitiendo a los estudiantes implementar la lógica de negocio sin la complejidad de integraciones externas.

Reglas generales

- Todo pago debe estar asociado a una reserva existente.
- No se puede registrar un pago para una reserva cancelada.
- El monto del pago debe ser mayor que cero.
- El pago debe corresponder al monto total de la reserva, no se permiten pagos parciales.
- Solo se puede realizar un pago por reserva.
- Una vez registrado el pago, la reserva debe cambiar automáticamente a estado confirmada.
- Cada transacción debe conservar su fecha, monto, medio de pago y un identificador único interno.

- Solo deben aceptarse medios de pago definidos por el sistema, en este caso, tarjeta de crédito simulada.

Reglas específicas para pago simulado

- El sistema debe permitir ingresar datos de tarjeta de crédito simulados (número, fecha de expiración y CVV).
- No se debe realizar validación real de los datos ingresados contra sistemas externos.
- El sistema debe asumir que todo pago es exitoso.
- No se deben simular errores de pago ni rechazos de transacción.
- El sistema debe registrar el pago como aprobado inmediatamente después de su confirmación.
- El usuario debe visualizar un resumen del pago antes de confirmarlo, incluyendo el monto total de la reserva.
- El sistema debe mostrar una confirmación clara del pago realizado.

3.2.6. Confirmación y seguimiento de reservas (Épica 6)

Una vez que la reserva ha sido registrada y se han efectuado los pagos correspondientes, la plataforma debe permitir gestionar el estado de dicha reserva y proporcionar visibilidad tanto al cliente como a la agencia.

El cliente debe poder acceder a su reserva, visualizar sus detalles, revisar el estado actual y obtener información relevante para su viaje. Esto puede incluir fechas, servicios contratados, cantidad de pasajeros y condiciones asociadas.

Por su parte, la agencia debe contar con herramientas para supervisar las reservas activas, realizar actualizaciones cuando sea necesario y gestionar cambios o cancelaciones. Esta actividad también puede incluir la generación de comprobantes, documentos o confirmaciones que respalden la operación.

Reglas de negocio

- Toda reserva debe tener un estado válido y controlado por el sistema.
- Una reserva solo puede marcarse como confirmada si cumple las condiciones establecidas, como pago completo.
- Un cliente solo puede visualizar sus propias reservas.
- Un administrador puede consultar y gestionar todas las reservas.
- Una reserva cancelada no debe aceptar nuevos pagos.
- El sistema debe permitir emitir comprobantes o constancias solo para reservas válidamente registradas.

3.2.7. Generación de reportes (Épica 7)

La plataforma debe permitir recopilar y procesar la información generada por las distintas operaciones realizadas en el sistema, con el fin de construir reportes que apoyen la gestión administrativa y la toma de decisiones de la agencia de viajes. Esta actividad se centra en transformar los datos operacionales, como reservas, pagos y paquetes turísticos, en información estructurada y útil para el análisis del negocio.

En este contexto, el sistema debe implementar al menos dos reportes principales. El primero corresponde al listado de ventas por período, el cual permite visualizar en detalle todas las ventas realizadas dentro de un rango de fechas definido por el usuario. Este reporte debe incluir información relevante como la fecha de la operación, el cliente asociado, el paquete turístico vendido, la cantidad de pasajeros, el monto total de la reserva, el monto pagado y

el estado de la misma. Su objetivo es entregar una visión detallada y cronológica de las transacciones realizadas en un período determinado.

El segundo corresponde al ranking de paquetes vendidos por período, el cual permite identificar los paquetes turísticos con mayor nivel de demanda dentro de un intervalo de tiempo específico. Este reporte debe consolidar la información de ventas agrupándola por paquete turístico, mostrando indicadores como cantidad de reservas, número total de pasajeros y, opcionalmente, el monto total generado por cada paquete. Su finalidad es facilitar el análisis comercial, permitiendo detectar tendencias, evaluar el desempeño de la oferta y apoyar la toma de decisiones estratégicas.

Ambos reportes deben poder generarse mediante la selección de un rango de fechas, garantizando que la información presentada sea consistente con los datos almacenados en el sistema. Asimismo, deben ser accesibles para usuarios con permisos administrativos y presentarse de forma clara para facilitar su interpretación.

Reglas generales

- El usuario debe ingresar una fecha de inicio y una fecha de término para generar los reportes.
- La fecha de inicio no puede ser posterior a la fecha de término.

Para el listado de ventas por período:

- Solo deben incluirse reservas cuya fecha de registro o pago esté dentro del período seleccionado.
- No deben incluirse reservas canceladas, salvo que el negocio lo requiera explícitamente.
- El reporte debe mostrar información consistente: cliente, paquete, cantidad de pasajeros, monto total y estado.
- Los montos deben reflejar los valores reales registrados en el sistema.

Para el ranking de paquetes vendidos por período:

- El ranking debe considerar únicamente las ventas dentro del período seleccionado.
- El sistema debe agrupar las ventas por paquete turístico.
- El criterio principal de ordenamiento debe ser definido, por ejemplo, cantidad de reservas o número de pasajeros.
- El ranking debe ordenarse de mayor a menor según el criterio definido.
- En caso de empate, se debe aplicar un criterio secundario consistente (por ejemplo, orden alfabético o monto total vendido).
- No deben considerarse reservas canceladas en el cálculo del ranking.

Reglas generales de acceso:

- Solo usuarios con rol administrativo pueden acceder a estos reportes.
- Los reportes deben reflejar la información actualizada al momento de la consulta.
- El sistema debe permitir generar los reportes bajo demanda cada vez que sean solicitados.

3.3 Resumen de funcionalidades a implementar

En esta sección se presenta un resumen de las funcionalidades que se necesitan implementar en cada microservicio. A continuación, se detalla cada una:

Detalle de funcionalidades	Microservicio
Épica 1. Gestión de usuarios y clientes	Keycloak (M1)
Épica 2. Publicación y gestión de paquetes turísticos	M2
Épica 3. Exploración y búsqueda de paquetes por parte del cliente	M3
Épica 4. Proceso de reserva en línea	M4
Épica 5. Gestión de pagos en línea	M5
Épica 6. Confirmación y seguimiento de reservas	M6
Épica 7. Generación de reportes	M7

4. Aspectos del desarrollo del producto

4.1 Respetto del Frontend

- Debe ser desarrollado usando ReactJS.
- Se requiere un único frontend para la aplicación.
- Se sugiere desarrollar usando *Visual Studio Code*.

4.2 Respetto del Backend

- Debe ser desarrollado usando el patrón arquitectural de microservicios.
- Debe ser desarrollado en *IntelliJ* o *Visual Studio Code*.
- Cada microservicio del backend debe ser desarrollado usando Spring Boot y usando una arquitectura de capas (@RestController, @Service, @Repository, y @Entity).
- El código fuente del backend debe ser escrito usando programación orientada a objetos.
- Cada microservicio (ver punto 3.3) debe ser implementado como un proyecto único e independiente.
- Cada microservicio debe usar su propia base de datos relacional (MySQL o PostgreSQL).
- El backend (además de tener los microservicios que implementan las Epicas) debe tener implementado los patrones de microservicios *ConfigServer*, *Service Discovery (Eureka Server)* y *API Gateway*.
- A excepción de *ConfigServer*, *Service Discovery (Eureka Server)* y *API Gateway*, todos los demás microservicios deben tener puertos asignados por el sistema (server.port=0).
- Los microservicios deben comunicarse entre sí (según corresponda) mediante peticiones HTTP (usando *RestTemplate*).

4.3 Despliegue de la aplicación web en producción

- El frontend y todos los microservicios del backend deben ser empaquetados en contenedores Docker independientes y luego almacenados en *Docker Hub*.
- El despliegue de la aplicación (tanto del backend como del frontend) se debe realizar hacia un cluster de *Kubernetes* (por ejemplo, *minikube*) desde las imágenes almacenadas en *Docker Hub*. Se deben usar scripts del tipo *Deployment*, *Service*, *ConfigMap*, *Secrets*, etc. para realizar este despliegue en producción. *Nota: El cluster de Kubernetes puede estar localmente.*
- La aplicación web debe poder ser accedida desde un navegador web.

- **IMPORTANTE:**

- No se acepta el uso de *port-forward* para redirigir el puerto local de la computadora hacia un Pod (o Service) dentro del cluster de Kubernetes (*minikube*).
- Minikube se debe levantar desde una Virtual Machine (Virtual Box, etc.).
- Se debe usar *ClusterIP* para exponer los microservicios (Pods) dentro del Cluster y así puedan comunicarse entre ellos.
- Se debe usar *NodePort* para exponer el API Gateway. Los microservicios dentro del Cluster no deben comunicarse entre ellos a través del API Gateway. Solo el Frontend es quien se comunica al API Gateway.